

74. Shadow DOM

Shadow DOM **encapsulation** ke liye hai. Component ka apna "shadow" DOM tree ho sakta hai jise main document se galti se access nahi kar sakte, aur uske apne local style rules ho sakte hain.

Built-in shadow DOM

Browser ke complex controls (jaise `<input type="range">`) andar DOM/CSS se hi draw hote hain. Wo structure hum se chhupa hota hai, par DevTools me dikhta hai (Chrome me "Show user agent shadow DOM" option on karo). `#shadow-root` ke neeche jo dikhta hai wahi **shadow DOM** hai.

Ise normal JS calls ya selectors se nahi pakad sakte. Ye normal children nahi, ek powerful encapsulation technique hai. Chrome me ek non-standard `pseudo` attribute dikhta hai jisse sub-elements ko CSS se style kar sakte hain:

CSS

```
input::-webkit-slider-runnable-track { background: red; }
```

Pehle browsers ne apne controls ke liye internal DOM structures banaye, baad me shadow DOM standardize hua taaki hum bhi wahi kar saken.

Shadow tree

Ek DOM element me 2 tarah ke subtrees ho sakte hain:

1. **Light tree**: normal DOM subtree, HTML children se bana. Ab tak ke saare chapters me "light" the.
2. **Shadow tree**: chhupa hua DOM subtree, HTML me reflect nahi hota.

Dono ho to browser **sirf shadow tree** render karta hai (light aur shadow ko compose karna baad me, slots chapter me).

Custom Elements me shadow tree component ke internals chhupane aur component-local styles lagane ke kaam aata hai:

js

```
customElements.define('show-hello', class extends HTMLElement {
  connectedCallback() {
    const shadow = this.attachShadow({mode: 'open'});
    shadow.innerHTML = `

Hello, ${this.getAttribute('name')}

`;
  }
});
```

html

```
<show-hello name="John"></show-hello>
```

`elem.attachShadow({mode: ...})` shadow tree banata hai. 2 limitations:

1. Ek element par sirf **ek** shadow root.
2. `elem` ya to custom element ho, ya inme se: `article`, `aside`, `blockquote`, `body`, `div`, `footer`, `h1..h6`, `header`, `main`, `nav`, `p`, `section`, `span`, `<img>` jaise elements shadow tree host nahi kar sakte.

`mode` encapsulation ka level batata hai:

- **"open"**: shadow root `elem.shadowRoot` se milta hai. Koi bhi code access kar sakta hai.
- **"closed"**: `elem.shadowRoot` hamesha `null`. Sirf `attachShadow` ke return reference se access (jo class ke andar chhupa rakhte hain). Browser ke native shadow trees (jaise `<input type="range">`) closed hote hain.

`attachShadow` ka return **shadow root** element jaisa hai: `innerHTML` ya `append` jaise DOM methods se bhar sakte ho. Shadow root wale element ko **shadow tree host** kehte hain, aur shadow root ki `host` property se milta hai:

js

```
alert(elem.shadowRoot.host === elem); // true (mode: "open" me)
```

Encapsulation

Shadow DOM main document se **kaafi alag** hai:

1. Shadow DOM ke elements light DOM ke `querySelector` ko **dikhte nahi**. Shadow DOM me light DOM ke ids se conflict hone wale ids bhi ho sakte hain (unique sirf shadow tree ke andar hone chahiye).
2. Shadow DOM ki **apni stylesheets** hoti hain. Bahar ke DOM ke style rules **lagte nahi**.

html

```

<style> p { color: red; } </style>    <!-- shadow tree par nahi lagega -->

<div id="elem"></div>
<script>
  elem.attachShadow({mode: 'open'});
  elem.shadowRoot.innerHTML = `
    <style> p { font-weight: bold; } </style>
    <p>Hello, John!</p>
  `;

  alert(document.querySelectorAll('p').length);           // 0
  alert(elem.shadowRoot.querySelectorAll('p').length);   // 1
</script>

```

Shadow tree ke elements dhundhne ke liye query tree ke **andar se** karni padti hai.

Summary

Shadow DOM component-local DOM banane ka tarika:

1. `shadowRoot = elem.attachShadow({mode: open|closed})`. `open` ho to `elem.shadowRoot` se milta hai.
2. `shadowRoot` ko `innerHTML` ya dusre DOM methods se bharo.

Shadow DOM ke elements:

- Ke apne **ids ka space**
- Main document ke selectors (`querySelector`) ko **invisible**
- Sirf **shadow tree ke styles** use karte hain, main document ke nahi

Shadow DOM ho to browser use ("light DOM" ki jagah) render karta hai. Dono ko kaise compose karein, ye agle chapter (slots) me.